Part 1- One Neuron, By Hand

1.1 First Calculation

Given:

$x_1 = 1$

$x_2 = 0$

$w_1 = 0.6$

$w_2 = -0.4$

Bias = -0.3

Calculation:

$z = (1 \times 0.6) + (0 \times -0.4) - 0.3$

$z = 0.6 + 0 - 0.3$

$z = 0.3$

Since 0.3 is greater than or equal to zero, the output is 1.

1.2 Second calculation Given: $x_1 = 0$ $x_2 = 1$. Calculation: $z = (0 \times 0.6) + (1 \times -0.4) - 0.3$ $z = 0 - 0.4 - 0.3$ $z = -0.7$ Since -0.7 is less than zero, the output is 0. 1.3 XOR explanation A single neuron cannot correctly solve XOR because XOR is not linearly separable. One neuron can only create a straight-line decision boundary. A neural network needs an additional hidden layer to combine multiple boundaries and solve this problem.

Four Attempts to Solve XOR Using One Neuron

The expected XOR outputs for inputs (0,0), (0,1), (1,0), and (1,1) are 0, 1, 1, and 0.

Attempt 1: w1 = 1, w2 = 1, bias = -0.5

Outputs: 0, 1, 1, 1. This failed because the last output should be 0.

Attempt 2: w1 = 1, w2 = -1, bias = -0.5

Outputs: 0, 0, 1, 0. This failed because the second output should be 1.

Attempt 3: w1 = -1, w2 = 1, bias = -0.5

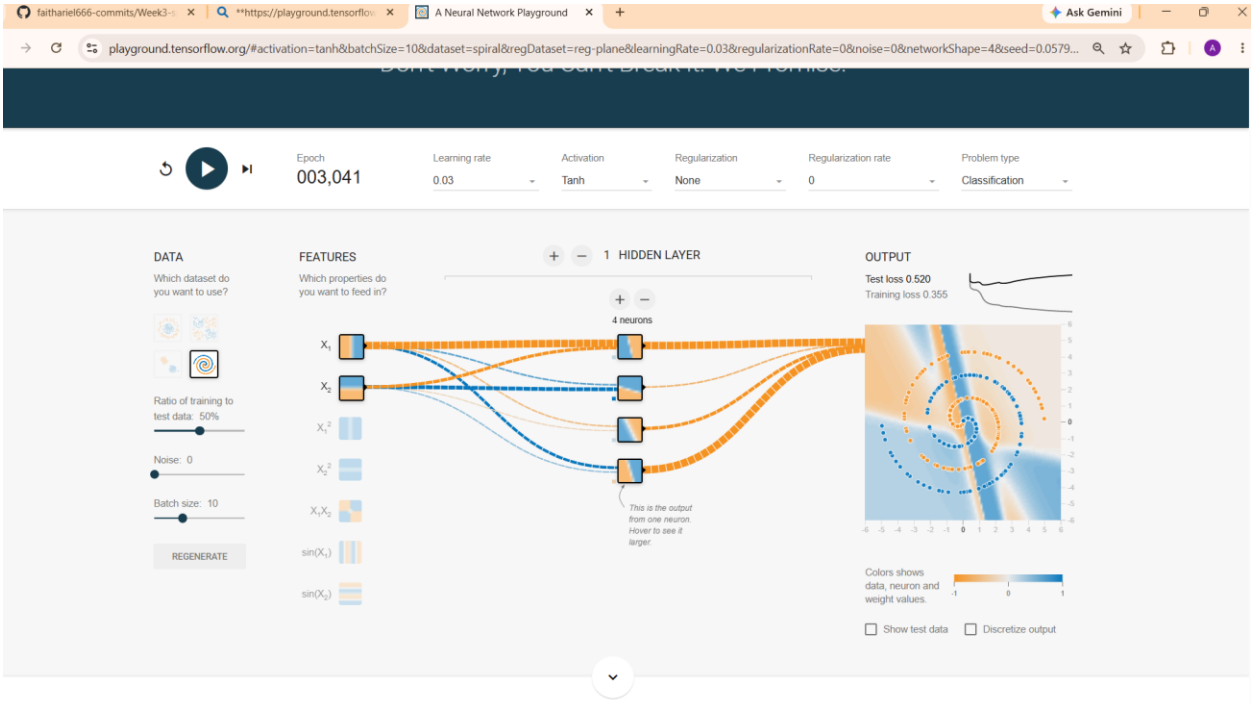
Outputs: 0, 1, 0, 0. This failed because the third output should be 1.

Attempt 4: w1 = 1, w2 = 1, bias = -1.5

Outputs: 0, 0, 0, 1. This failed because the second and third outputs should be 1.

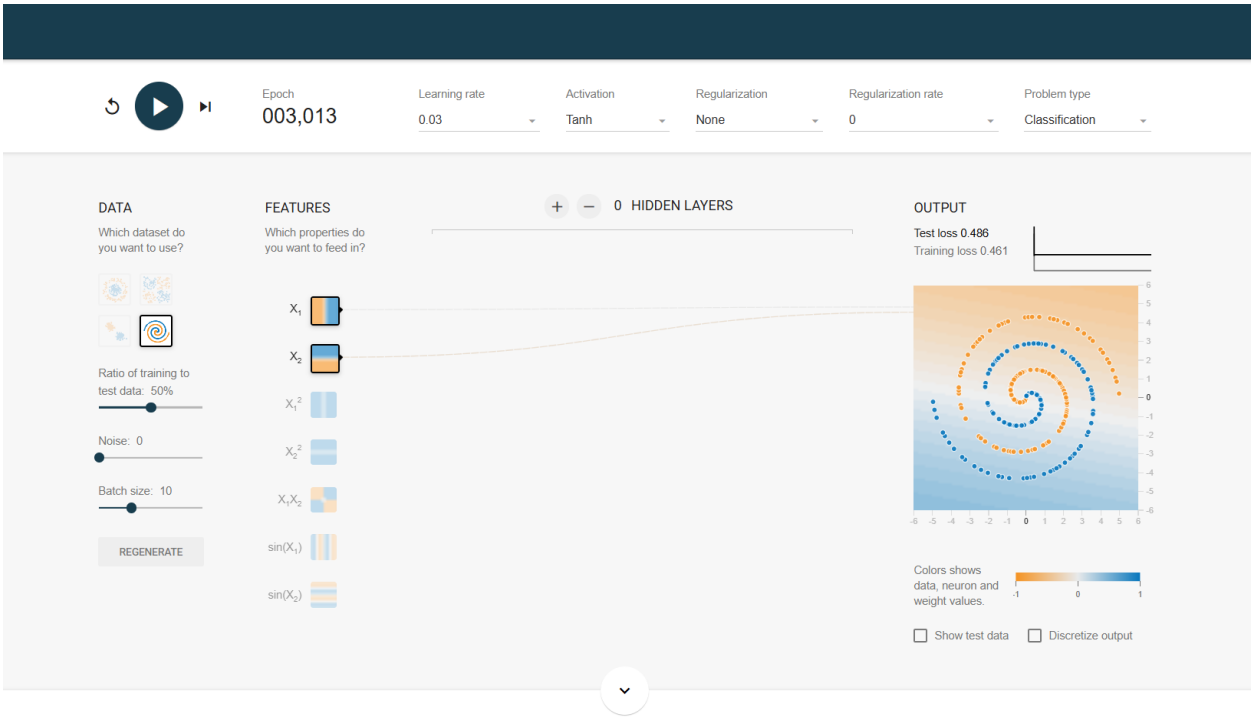
Conclusion: None of these attempts correctly solved XOR. A single neuron cannot solve XOR because it can only create a straight-line decision boundary. A neural network needs an additional hidden layer to solve this problem.

EXPERIMENT 1: Spiral Dataset with One Hidden Layer

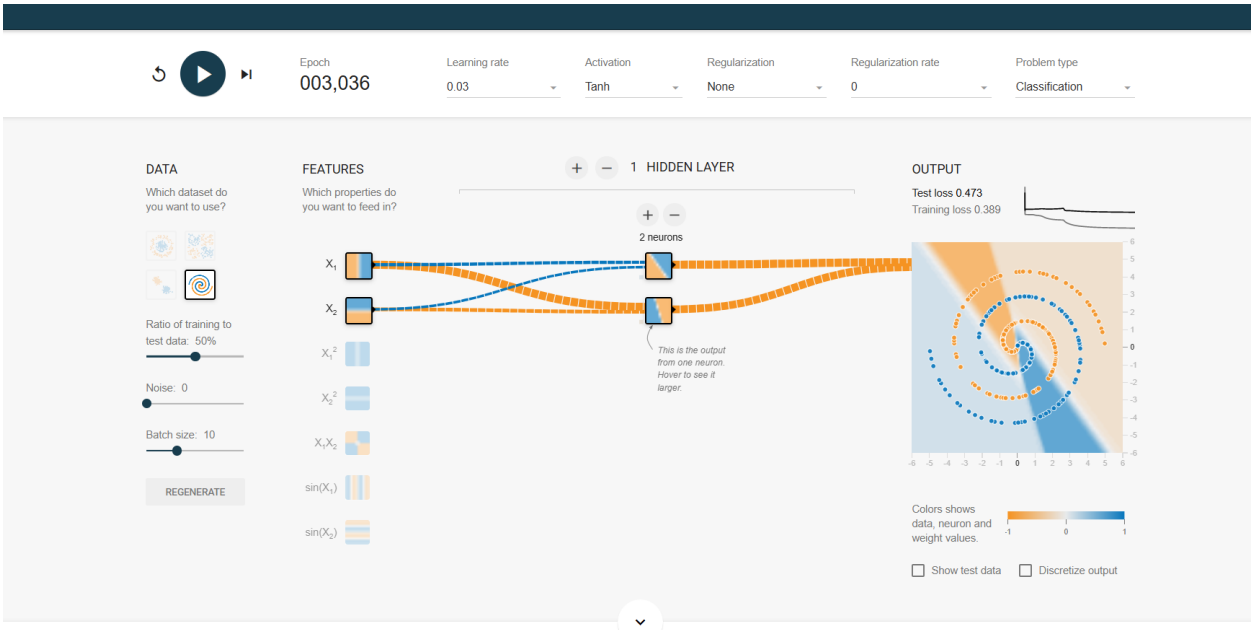


The spiral dataset was difficult for a neural network with only one hidden layer to classify correctly. Adding more hidden layers and neurons helped the network learn the complex spiral pattern. The hidden layers allowed the network to create more complex decision boundaries and separate the two classes more effectively.

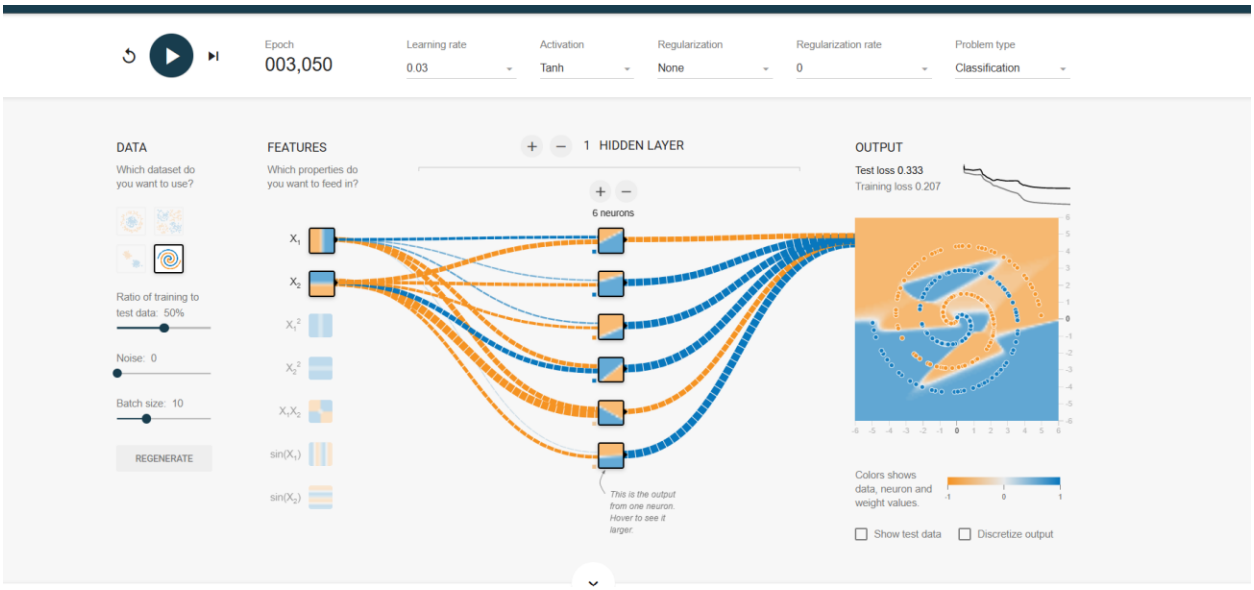
Experiment 2: Spiral dataset with zero hidden layers.



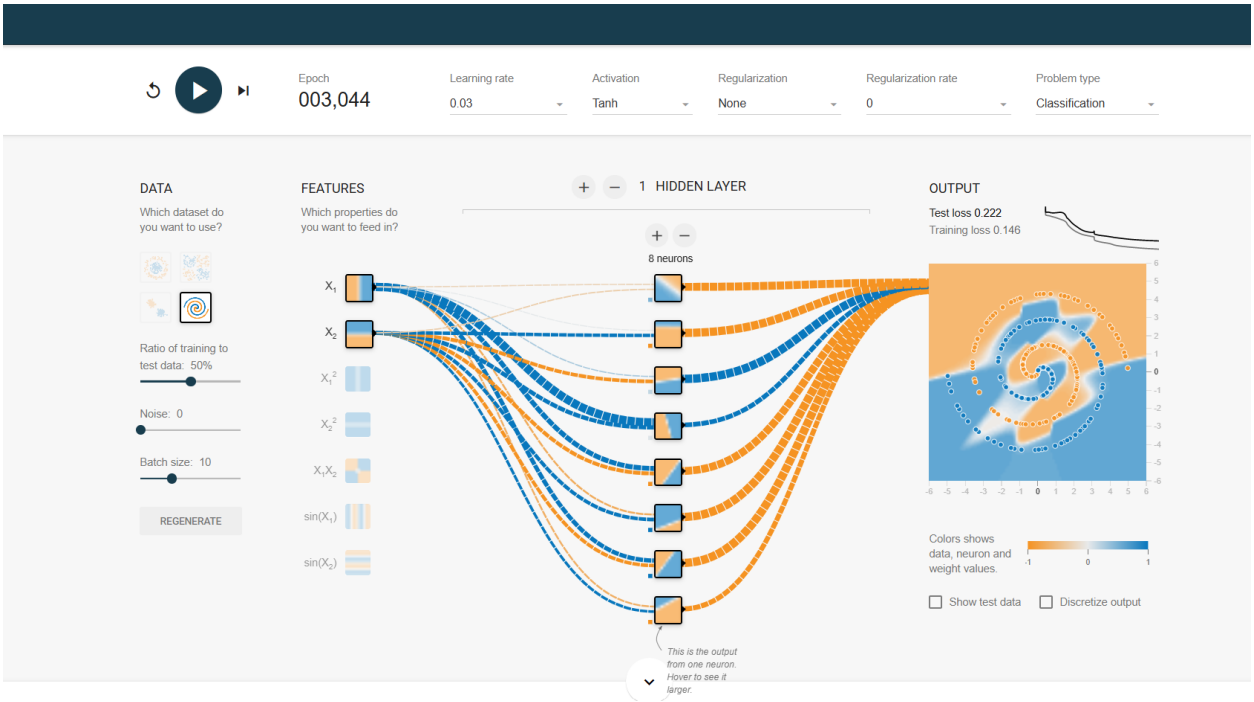
One Hidden layer with two neurons



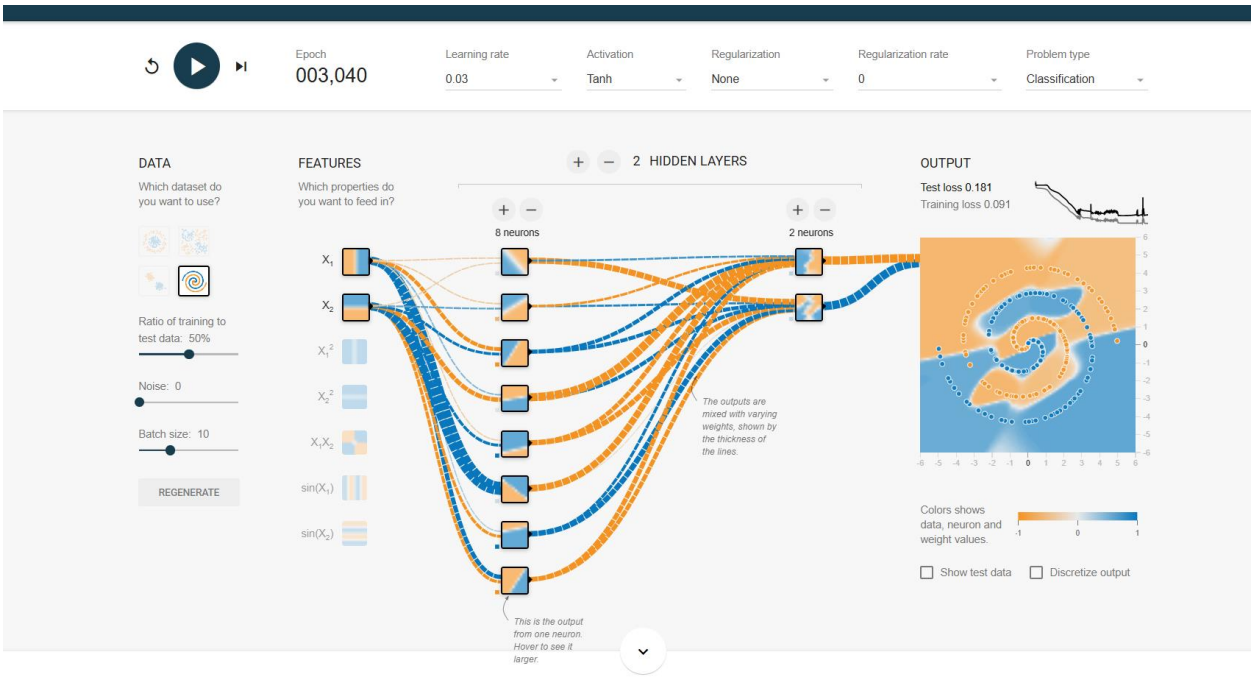
Experiment 4: One hidden layer with six neurons



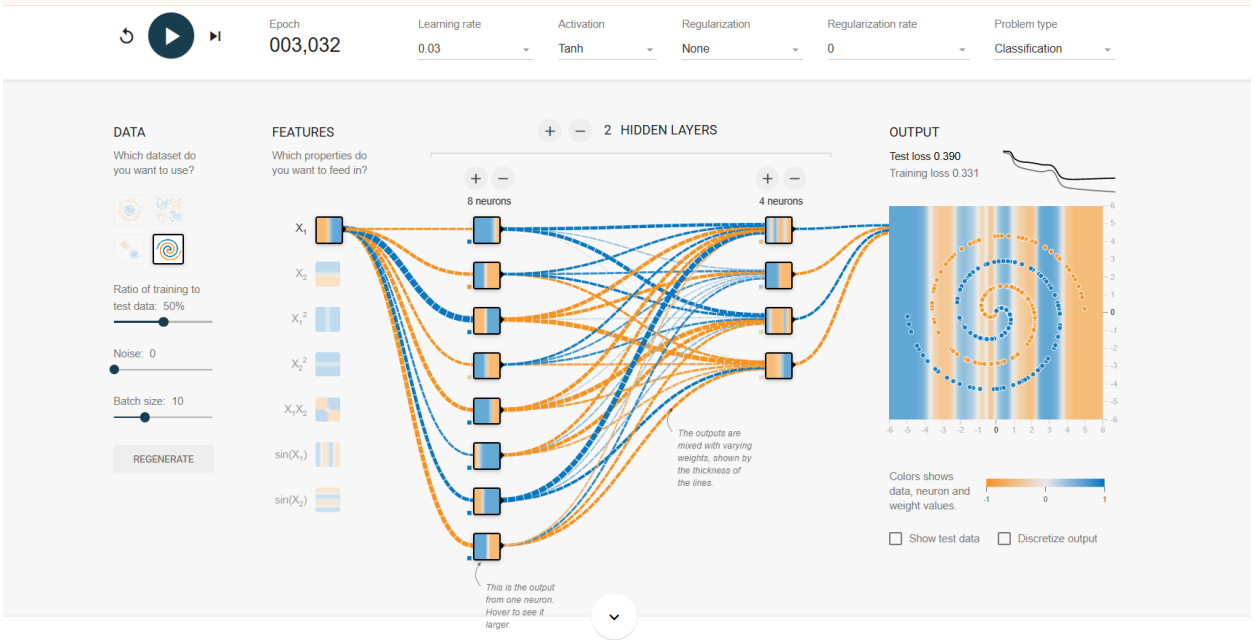
Experiment 5: one hidden layer with 8 neurons.



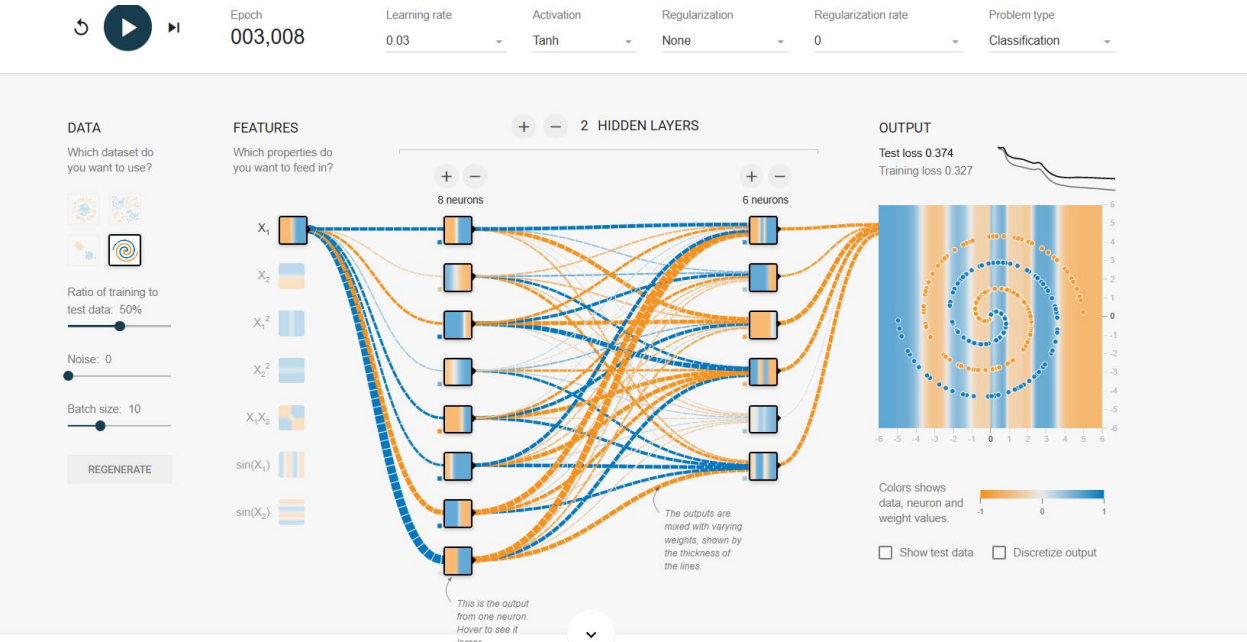
Experimen 6: Two hidden layer with 8 neurons in the first layer and 2 neurons in the second layer



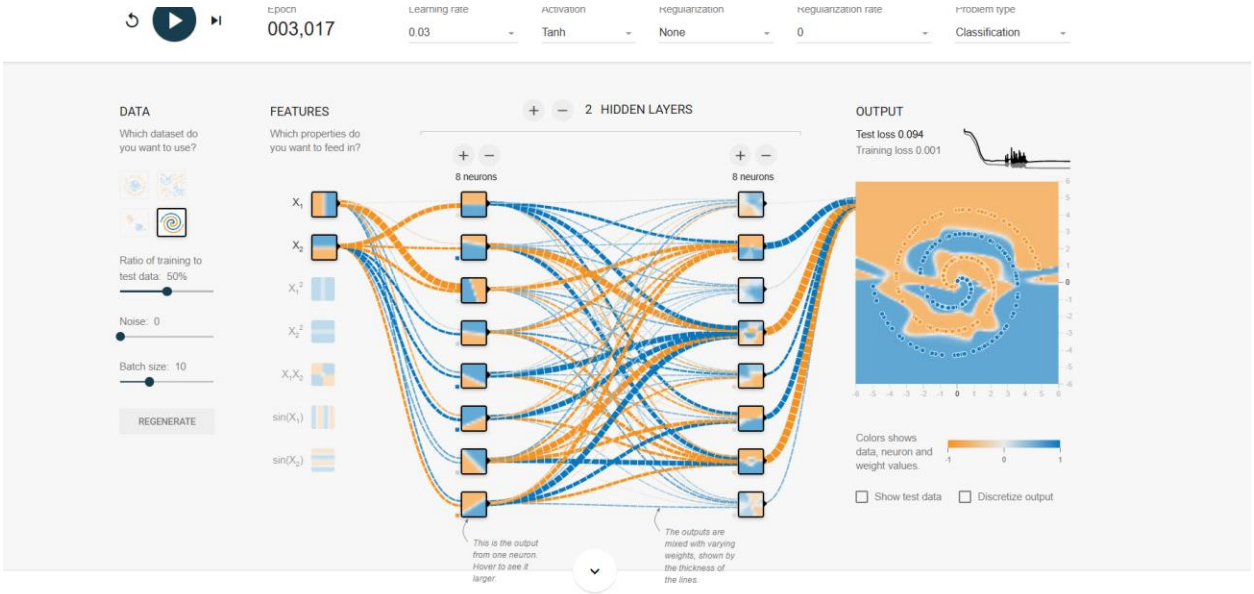
Experiment 7: Two hidden layer with 8 neurons in the first and 4 neurons in the second layer.



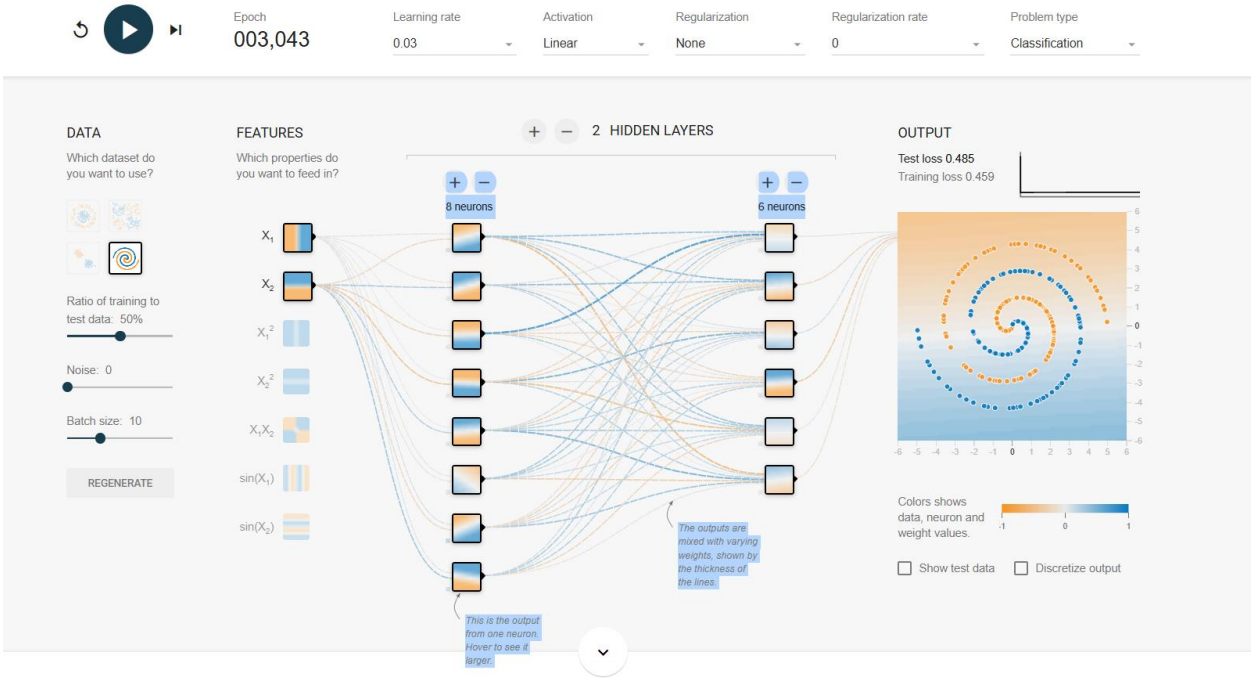
Experiment 8: Two hidden layers with 8 neurons in the first layer and 6 neurons in the second layer.



Experiment: Two hidden layers with 8 neurons in each layer, using both input features.

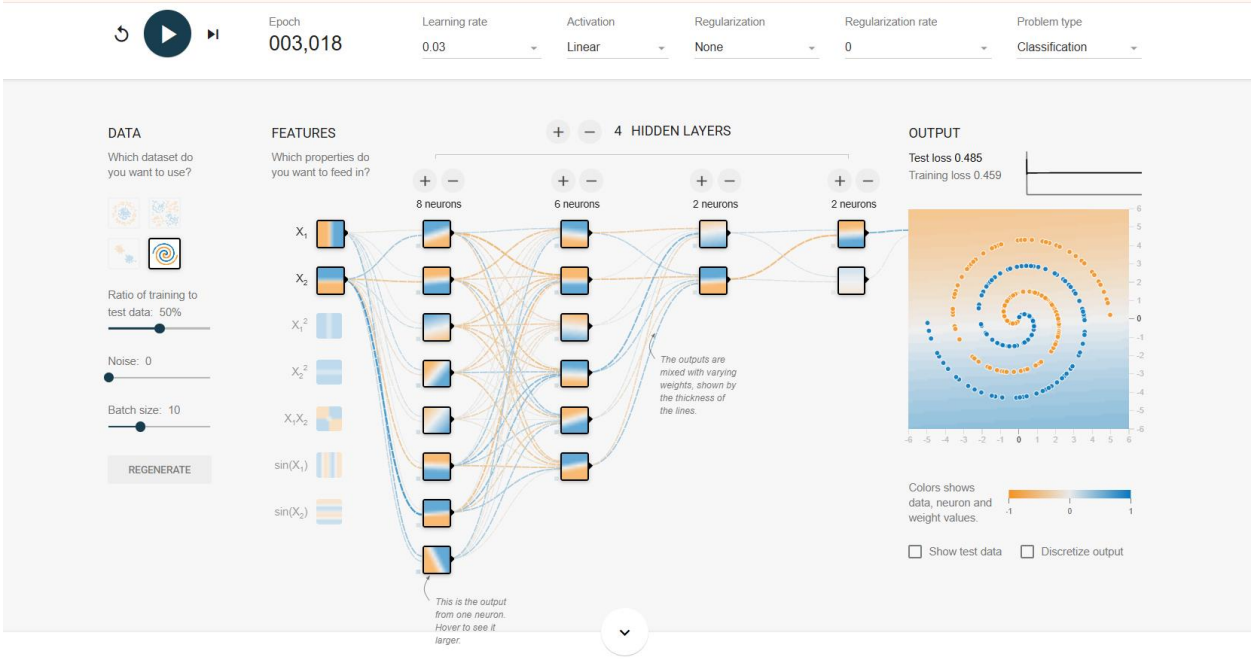


Um, What Is a Neural Network?

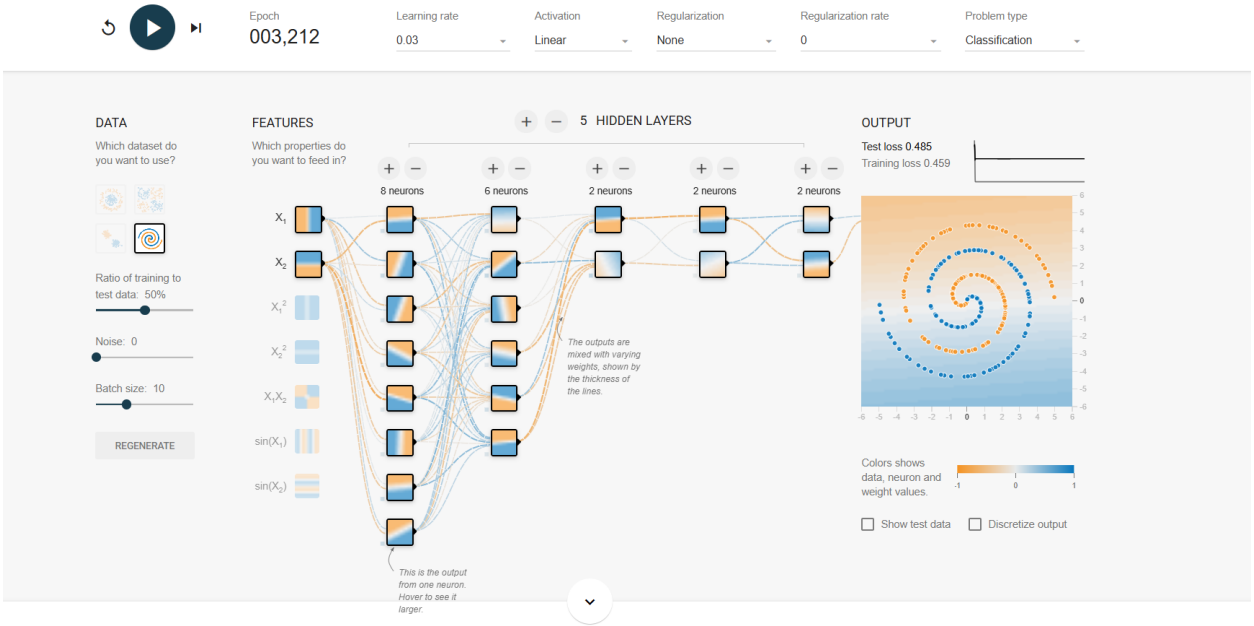


Um, What Is a Neural Network?

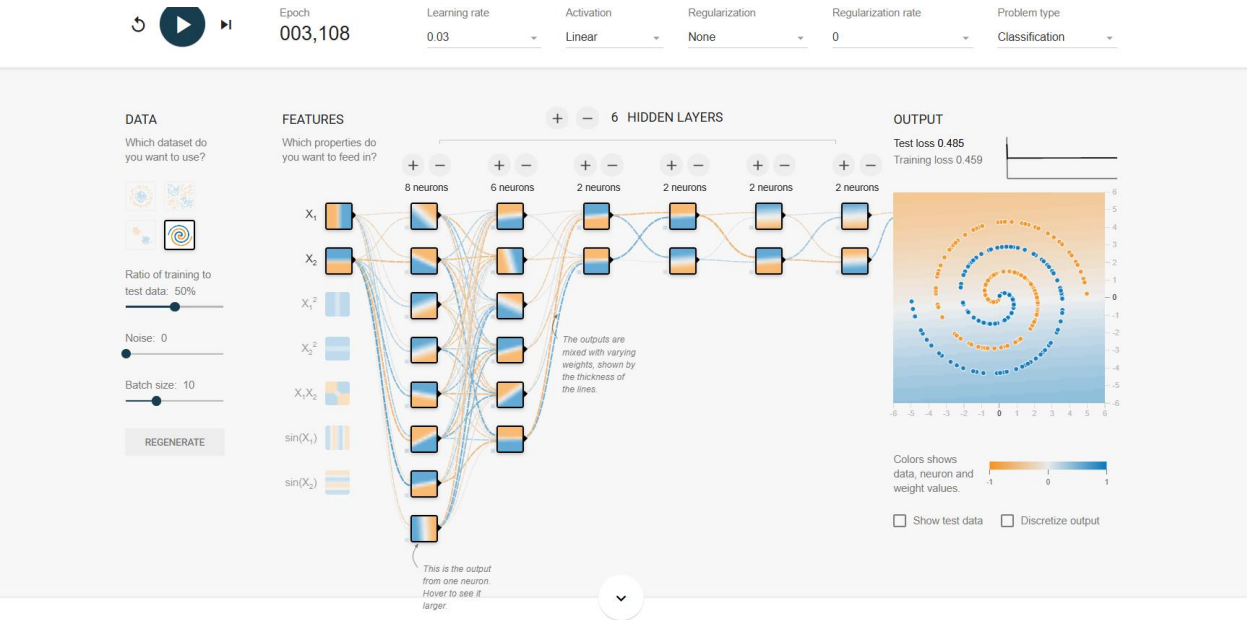
g



Um, What Is a Neural Network?



Um, What Is a Neural Network?



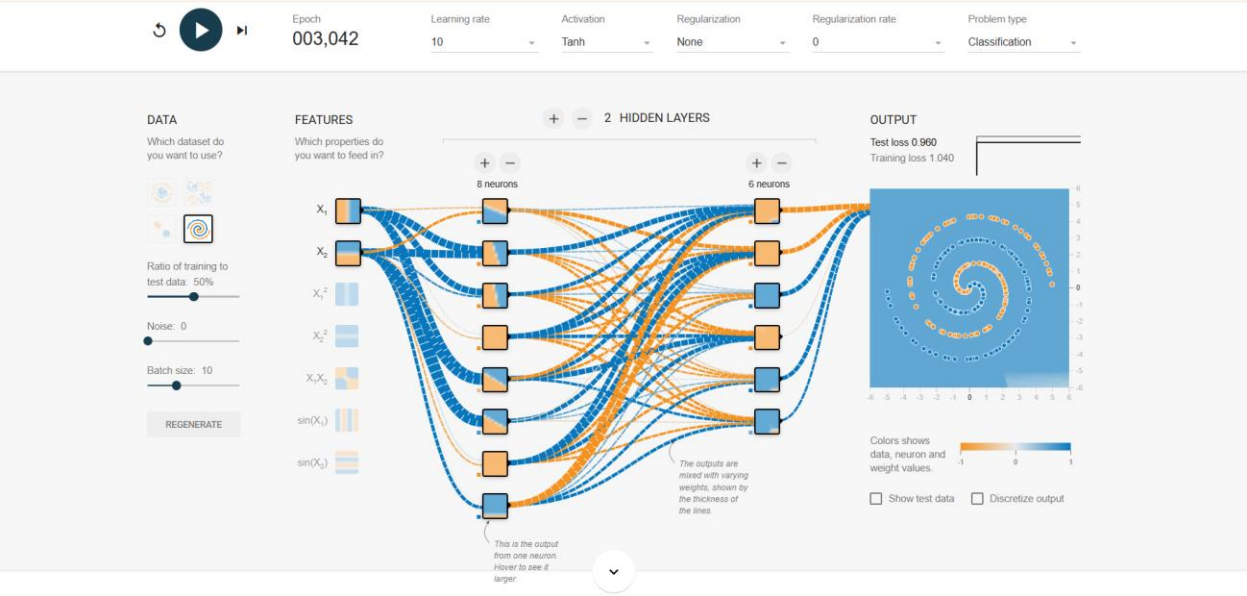
Part 2.2 – Linear Activation

I tested the neural network using Linear activation with 2, 4, 5, and 6 hidden layers. Adding more layers did not significantly improve the results. The network could not properly separate the spiral dataset.

This happens because stacking linear operations still produces a linear function. Therefore, even a deep network cannot learn complex patterns without a nonlinear activation function. Nonlinear activation is what makes hidden layers powerful.

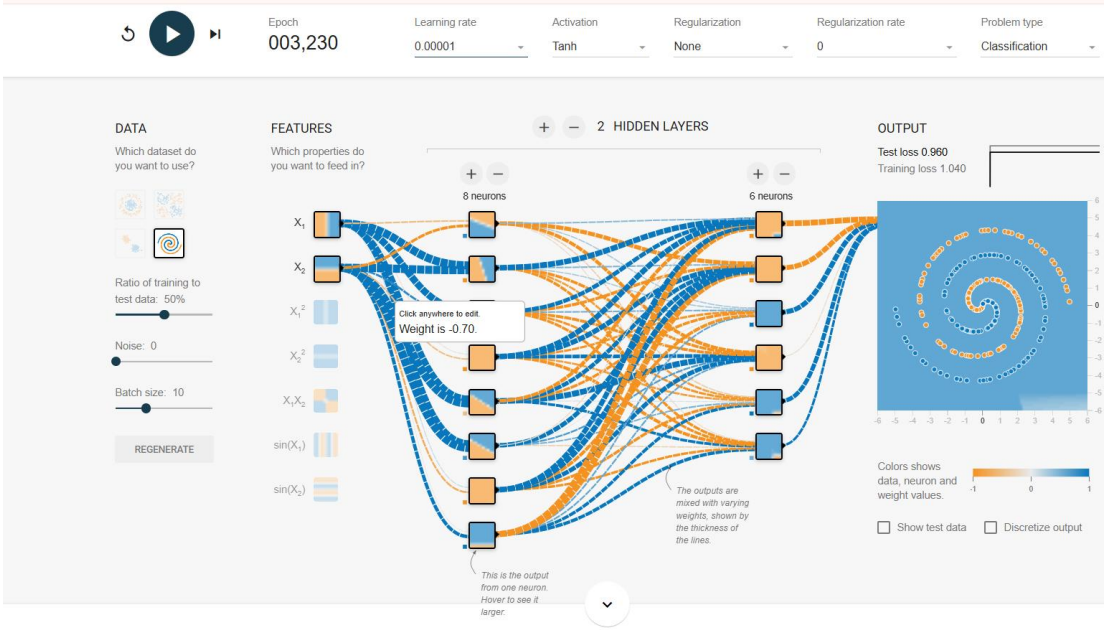
Experiment with a learing rate of 10, showing poor neural network performance

h



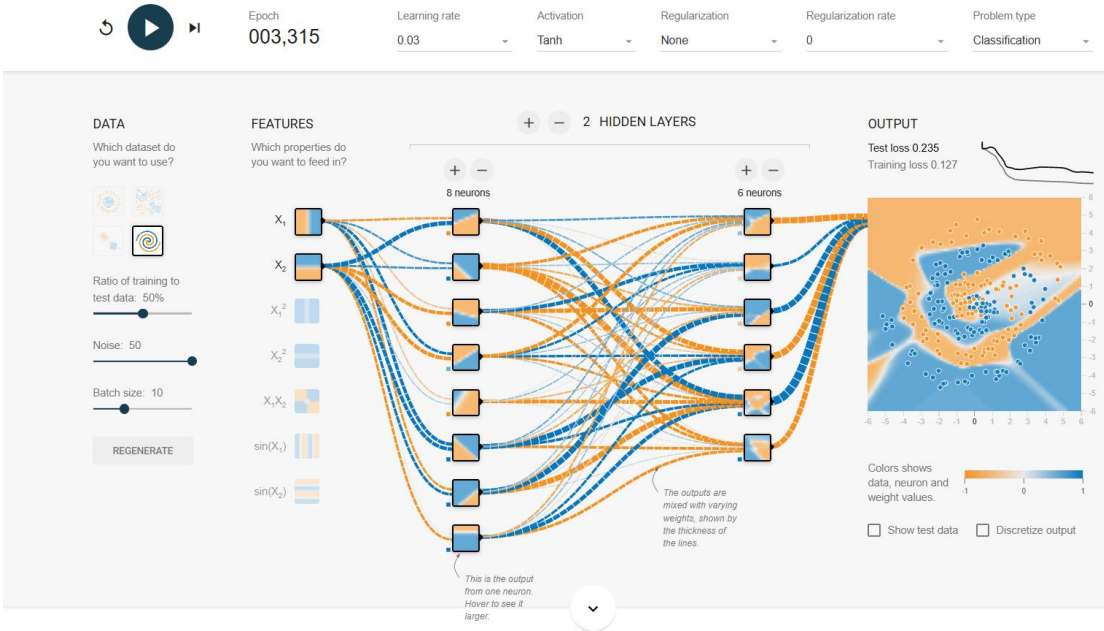
Um, What Is a Neural Network?

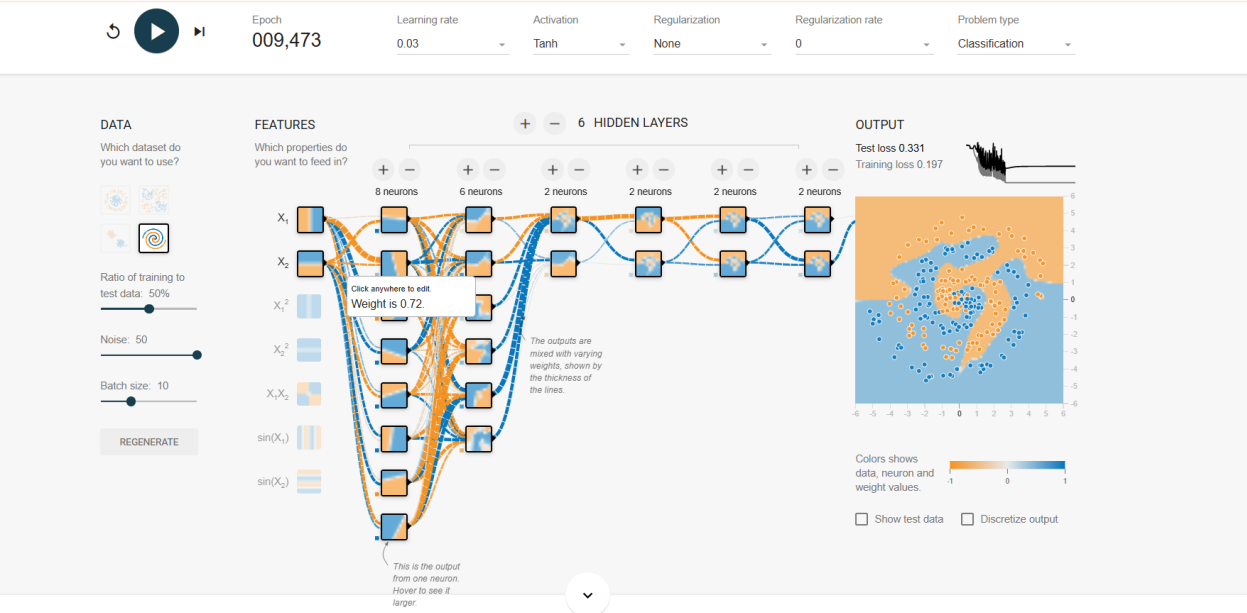
Experiment using a very low learning rate (0.00001). The network learns extremely slowly and struggles to improve its performance.



Um, What Is a Neural Network?

It's a technique for building a computer program that learns from data. It

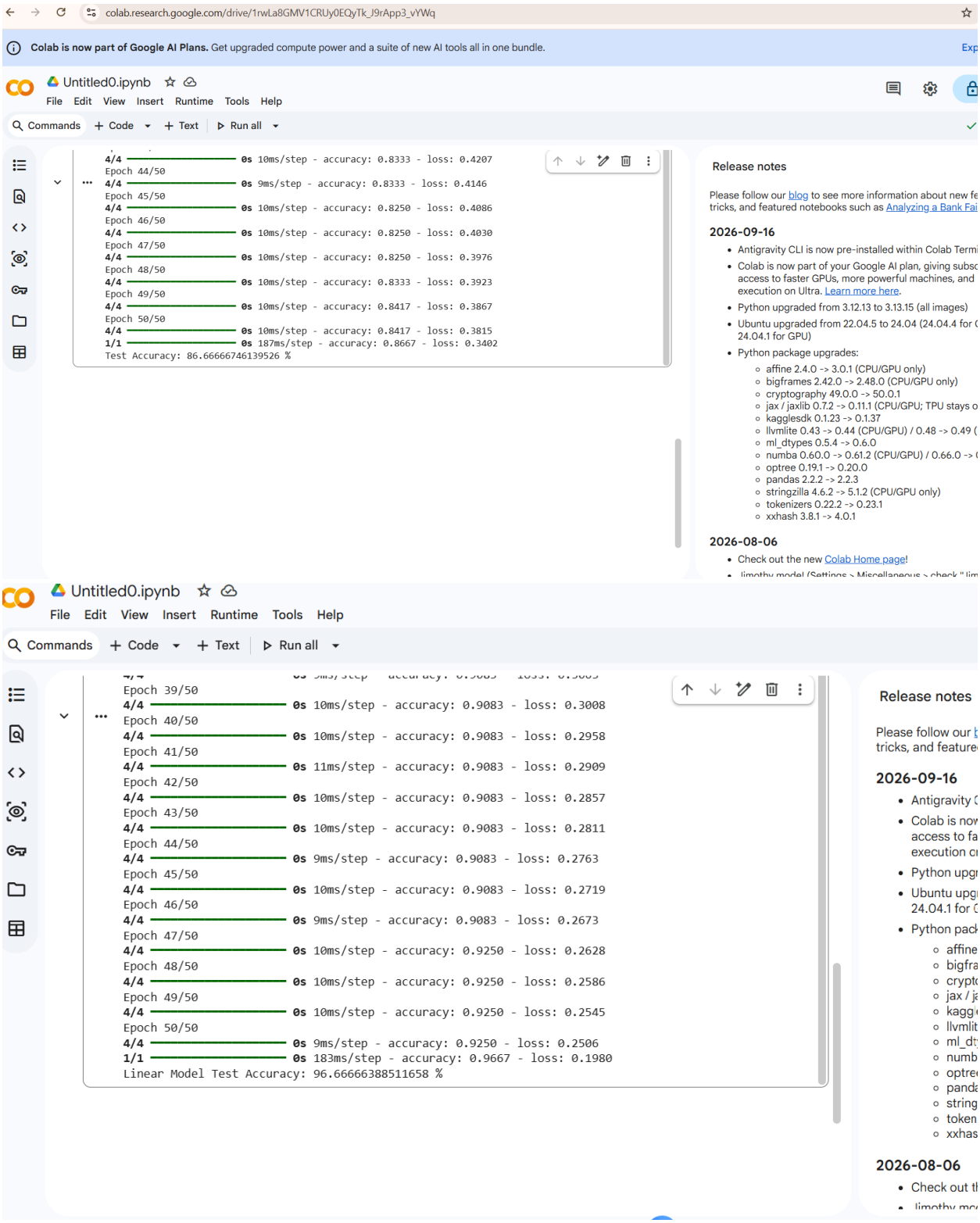




Um, What Is a Neural Network?

Figure: Neural network with six hidden layers trained on a noisy spiral dataset, showing a complex decision boundary.

The decision boundary has a complex, spiral-like shape. The network is learning the patterns in the training data, including random noise. This is overfitting because it may learn unnecessary details that do not help it classify new data correctly. The higher test loss compared with training loss shows that the model performs worse on unseen data.


colab.research.google.com/drive/1rwLa8GMV1CRUy0EQyTk_J9rApp3_vYWq
Colab is now part of Google AI Plans. Get upgraded compute power and a suite of new AI tools all in one bundle.
Untitled0.ipynb
File Edit View Insert Runtime Tools Help
Commands + Code + Text Run all
4/4 0s 10ms/step - accuracy: 0.8333 - loss: 0.4207
Epoch 44/50
4/4 0s 9ms/step - accuracy: 0.8333 - loss: 0.4146
Epoch 45/50
4/4 0s 10ms/step - accuracy: 0.8250 - loss: 0.4086
Epoch 46/50
4/4 0s 10ms/step - accuracy: 0.8250 - loss: 0.4030
Epoch 47/50
4/4 0s 10ms/step - accuracy: 0.8250 - loss: 0.3976
Epoch 48/50
4/4 0s 10ms/step - accuracy: 0.8333 - loss: 0.3923
Epoch 49/50
4/4 0s 10ms/step - accuracy: 0.8417 - loss: 0.3867
Epoch 50/50
4/4 0s 10ms/step - accuracy: 0.8417 - loss: 0.3815
1/1 0s 187ms/step - accuracy: 0.8667 - loss: 0.3402
Test Accuracy: 86.66666746139526 %
Release notes
Please follow our blog to see more information about new fe
tricks, and featured notebooks such as Analyzing a Bank Fai
2026-09-16
Antigravity CLI is now pre-installed within Colab Termi
Colab is now part of your Google AI plan, giving subsc
access to faster GPUs, more powerful machines, and
execution on Ultra. Learn more here.
Python upgraded from 3.12.13 to 3.13.15 (all images)
Ubuntu upgraded from 22.04.5 to 24.04 (24.04.4 for
24.04.1 for GPU)
Python package upgrades:
affine 2.4.0 -> 3.0.1 (CPU/GPU only)
bigframes 2.42.0 -> 2.48.0 (CPU/GPU only)
cryptography 49.0.0 -> 50.0.1
jax / jaxlib 0.7.2 -> 0.11.1 (CPU/GPU; TPU stays o
kagglesdk 0.1.23 -> 0.1.37
llvmlite 0.43 -> 0.44 (CPU/GPU) / 0.48 -> 0.49 (
ml_dtypes 0.5.4 -> 0.6.0
numba 0.60.0 -> 0.61.2 (CPU/GPU) / 0.66.0 ->
optree 0.19.1 -> 0.20.0
pandas 2.2.2 -> 2.2.3
stringzilla 4.6.2 -> 5.1.2 (CPU/GPU only)
tokenizers 0.22.2 -> 0.23.1
xxhash 3.8.1 -> 4.0.1
2026-08-06
Check out the new Colab Home page!
Untitled0.ipynb
File Edit View Insert Runtime Tools Help
Commands + Code + Text Run all
Epoch 39/50
4/4 0s 10ms/step - accuracy: 0.9083 - loss: 0.3008
Epoch 40/50
4/4 0s 10ms/step - accuracy: 0.9083 - loss: 0.2958
Epoch 41/50
4/4 0s 11ms/step - accuracy: 0.9083 - loss: 0.2909
Epoch 42/50
4/4 0s 10ms/step - accuracy: 0.9083 - loss: 0.2857
Epoch 43/50
4/4 0s 10ms/step - accuracy: 0.9083 - loss: 0.2811
Epoch 44/50
4/4 0s 9ms/step - accuracy: 0.9083 - loss: 0.2763
Epoch 45/50
4/4 0s 10ms/step - accuracy: 0.9083 - loss: 0.2719
Epoch 46/50
4/4 0s 9ms/step - accuracy: 0.9083 - loss: 0.2673
Epoch 47/50
4/4 0s 10ms/step - accuracy: 0.9250 - loss: 0.2628
Epoch 48/50
4/4 0s 10ms/step - accuracy: 0.9250 - loss: 0.2586
Epoch 49/50
4/4 0s 10ms/step - accuracy: 0.9250 - loss: 0.2545
Epoch 50/50
4/4 0s 9ms/step - accuracy: 0.9250 - loss: 0.2506
1/1 0s 183ms/step - accuracy: 0.9667 - loss: 0.1980
Linear Model Test Accuracy: 96.66666388511658 %
Release notes
2026-09-16
2026-08-06

```
plt.title('Training Loss vs Validation Loss')
plt.legend()
plt.grid(True)
plt.show()
```



The graph shows overfitting because the training loss decreases toward zero while the validation loss increases. This means the neural network is learning the training data very well but performing poorly on new data. The two lines begin to separate early in training, showing that the model is overfitting. I should have stopped training when the validation loss started increasing instead of allowing the model to continue for 500 epochs.

Explain backpropagation

Backpropagation is the process a neural network uses to learn from its mistakes. First, the network makes a prediction. It compares the prediction with the correct answer to find the error. The error is then sent backward through the network to adjust the weights in the right direction. This process repeats until the network becomes better at making accurate predictions.

One thing I believed that was wrong

Before this assignment, I believed that adding more hidden layers would always make a neural network perform better. However, I learned that deeper networks do not always improve performance. The activation function is important because it allows a network to learn complex patterns. I also learned that too much training can cause overfitting.